C++ Language Traps

Declaration rules, startup dependencies, proxies, deduction and concurrent access, with explicit version and evidence scope.

Debug build:
`-std=c++17 -Wall -Wextra -g -fsanitize=address,undefined`
Also try: `-O2 · -fsanitize=thread`

bug example (see captured check) · fixed example (checks have bounded scope) · ▶ = what gcc (GCC) 16.2.1 20260810 / its sanitizers actually reported · 11 entries

CPP-TRAP-01 Most vexing parse

SR-404 · Wiper timer

✕ BUGGY

```
Timer t();         // declares a FUNCTION returning Timer
t.ms = 500;        // error: 't' is not a class object
```

▶ `gcc -Wvexing-parse: empty parentheses were disambiguated as a function…`

What goes wrong: In this declaration context Timer t() declares a function returning Timer, so t.ms is invalid. Engine e(Timer()) declares a function returning Engine with a parameter adjusted to a pointer to a function returning Timer. The language has specific ambiguity-resolution rules, not a universal rule for every token sequence.

> FIX

```
Timer t{};         // value-initialised object (or just: Timer t;)
t.ms = 500;
```

Avoid it: Author guidance: use an unambiguous object initializer when an object is intended. Braces can choose initializer_list overloads or reject narrowing, so they are not interchangeable with every parenthesized initializer. GCC14 documents -Wvexing-parse; warning history and other compiler defaults need their own evidence.

Sources: C++17 draft 9.8 [stmt.ambig]; 11.2 [dcl.ambig.res]; 11.3.5 [dcl.fct]; GCC14 C++ options -Wvexing-parse

CPP-TRAP-02 Static initialisation order fiasco

SR-404 · Speedometer calibration

✕ BUGGY

```
// units.cpp
int mph_factor = read_calibration(); // may initialize dynamically

// gauges.cpp
extern int mph_factor;
int g_scale = 2 * mph_factor;  // may observe only zero initialization
```

▶ `ASan: initialization-order-fiasco`

What goes wrong: The shown ordinary globals are in different translation units and lack a single source-order guarantee across them. Static/zero initialization precedes dynamic initialization; ordering, permitted early initialization and deferral have separate N4659 rules. An early read can observe zero here; a physical crash for another class is not a required result.

> FIX

```
// units.h: keep runtime calibration, initialized at first use
int read_calibration();
inline int mph_factor() {
    static const int f = read_calibration();
    return f;
}
// gauges.cpp: evaluate after runtime calibration is ready
int gauge_scale() { return 2 * mph_factor(); }
```

Avoid it: Author guidance: remove hidden startup dependencies. The local-static accessor initializes on first passage and supplies the fixed calibration16 before multiplication. Initialization exceptions are retried on later passage; recursive reentry during initialization is invalid. ASan's captured initialization-order report is an instrumented host observation, not a universal C++ outcome.

Sources: C++17 draft 6.6.2 [basic.start.static]; 6.6.3 [basic.start.dynamic]; 9.7 [stmt.dcl]; Clang docs Initialization order checking; Limitations

CPP-TRAP-03 ODR violation: same inline name, different body

SR-404 · Speedometer vs. tachometer

✕ BUGGY

```
// speedo.cpp
inline int scale() { return 1; }     // km/h per tick

// tacho.cpp
inline int scale() { return 100; }   // rpm: same name, other body
```

▶ `compiled/linked rc=0; not executed: ODR mismatch: no diagnostic requir…`

What goes wrong: These two external inline definitions name the same entity but use different tokens, violating N4659's multiple-definition conditions. That cross-unit case requires no diagnostic. Token identity alone is not sufficient: name lookup and other ODR conditions also matter. The invalid program has no promised linker choice or output.

> FIX

```
// speedo.cpp
namespace { int scale() { return 1; } }     // internal linkage

// tacho.cpp
namespace { int scale() { return 100; } }   // a different function
```

Avoid it: Author guidance: use internal linkage for genuinely independent file-local helpers, and maintain consistent tokens and required lookup bindings for shared definitions. The bad example is compiled/linked only, with its raw result retained; it is not executed to invent a portable outcome. The fixed independent helpers are actually run.

Sources: C++17 draft 6.2 [basic.def.odr]p6; 10.3.1 [namespace.def]

CPP-TRAP-04 Implicit conversion via a non-explicit constructor

SR-404 · Rev limiter

✕ BUGGY

```
struct Rpm {
    Rpm(int v) : value(v) {}      // converting constructor
    int value;
};
void set_redline(Rpm r) { std::printf("redline %d\n", r.value); }
// set_redline(true);  compiles: bool → int → Rpm{1}
```

▶ `prints: redline 1`

What goes wrong: The non-explicit int constructor permits a standard bool-to-int conversion followed by construction of Rpm; the supplied true input produces value1. This demonstrates an admitted conversion, not validated rev-limit units or sensor range. Constructor/conversion behavior depends on the actual overloads and initialization context.

> FIX

```
struct Rpm {
    explicit Rpm(int v) : value(v) {}
    int value;
};
void set_redline(Rpm r) { std::printf("redline %d\n", r.value); }
// set_redline(true);       error: no implicit conversion
// set_redline(Rpm{7000});  intent is visible
```

Avoid it: Author guidance: choose explicit constructors for APIs where implicit conversion is unintended. Explicit still permits direct initialization and does not validate integer range; Rpm{true} remains possible here. Conversion to bool has contextual exceptions for explicit conversion functions. Conditional explicit is a C++20 facility, not this C++17 fix.

Sources: C++17 draft 15.3.1 [class.conv.ctor]; 7.14 [conv.bool]; 16.3.3.1 [over.best.ics]; 7 [conv]p4; 15.3.2 [class.conv.fct]; C++20 draft N4861 [dcl.fct.spec]: conditional explicit

CPP-TRAP-05 std::vector<bool> is not a container of bool

SR-404 · Dash warning lamps

✕ BUGGY

```
std::vector<bool> lamps(4);
bool& first = lamps[0];  // cannot bind this proxy to nonconst bool&
```

▶ `gcc error: cannot bind non-const lvalue reference of type 'bool&' to a…`

What goes wrong: N4659 vector<bool>::reference is a proxy, so the shown nonconst bool& binding is invalid. auto x=v[0] keeps that proxy; bool b=v[0] copies a value. Packed bits are recommended, not a mandated allocator layout. The standard excludes vector<bool> from the different-element concurrent-modification guarantee; not every pair of writes must race.

> FIX

```
std::vector<char> lamps(4);   // real, addressable elements
char& first = lamps[0];       // an addressable char element
first = 1;
```

Avoid it: Author guidance: use an addressable element type such as char when real element references are needed. No data() member is specified for this N4659 specialization. A proxy must still obey backing-storage validity; use an explicit bool value when a snapshot is wanted. bitset has its own proxy behavior and is not a universal bool-reference replacement.

Sources: C++17 draft 26.3.12 [vector.bool]; 26.2.2 [container.requirements.dataraces]; 23.9.2 [template.bitset]

CPP-TRAP-06 auto deduction surprises

SR-404 · Trip computer tables

✕ BUGGY

```cpp
const std::vector<int>& readings();  // this fixture has three elements
void scan() {
    auto r = readings();   // std::vector<int>: a full COPY
    auto x = {5};          // std::initializer_list<int>, not int
    std::cout << r.size() + x.size() << '\n';
}
```

▸ prints: 4

What goes wrong: For this ordinary by-value auto declaration, deduction removes the initializer's reference/top-level const and copies the lvalue vector. Other initializers can move or elide. Under N4659 auto x={5} deduces initializer_list<int>, while auto y{5} deduces int. N3922 is defect wording, not proof that every earlier language mode must differ.

✓ FIX

```cpp
const std::vector<int>& readings();  // this fixture has three elements
void scan() {
    const auto& r = readings();  // binds to the table: no copy
    auto x = 5;  // int (auto x{5} is also int in C++17)
    std::cout << r.size() + x << '\n';
}
```

Avoid it: Author guidance: spell the intended reference/value policy and check deduced types. The fixed const auto& borrows the live static table; its lifetime is part of the contract. Modern compilers may apply defect changes in earlier modes. Avoiding every brace is a style choice, not a language requirement; the actual hidden type checks are called in this candidate.

Sources: C++17 draft 10.1.7.4 [dcl.spec.auto]; 10.1.7.4.1 [dcl.type.auto.deduct]; WG21 N3922 (defect wording) Background; Summary of Proposed Changes

CPP-TRAP-07 Lambda captures a local by reference, then outlives it

SR-404 · Tachometer callback

✕ BUGGY

```cpp
std::function<int()> make_reader() {
    int rpm = 3000;
    return [&] { return rpm; };   // rpm dies on return
}
```

▸ ASan: stack-use-after-return

What goes wrong: The returned lambda refers to the automatic int rpm after its lifetime ends. Invoking that access is invalid; the language does not mandate a physical dead stack slot or particular printed value. A lambda can outlive a capture's declaring scope safely when the referenced backing object remains live.

✓ FIX

```cpp
std::function<int()> make_reader() {
    int rpm = 3000;
    return [rpm] { return rpm; };  // copy into the closure
}
```

Avoid it: Author guidance: choose capture ownership deliberately. Copying a pointer/view does not extend its backing lifetime. A move-only init-capture can make the closure move-only; N4659 std::function requires a copy-constructible target. The shown captured int is copyable. ASan's return-lifetime capture has instrumentation/platform scope.

Sources: C++17 draft 8.1.5.2 [expr.prim.lambda.capture]; 6.8 [basic.life]; 23.14.13.2.1 [func.wrap.func.con]; C++ Core Guidelines (guidance) F.53 (design guidance); Clang docs Stack use after return; Limitations

CPP-TRAP-08 Data race on a shared counter

SR-404 · Wheel-tick counter

✕ BUGGY

```cpp
unsigned long ticks = 0;
std::thread t([&] { for (unsigned long i = 0; i < 100000UL; ++i) ++ticks; });
for (unsigned long i = 0; i < 100000UL; ++i) ++ticks;  // unsynchronised: UB
t.join();
std::cout << ticks << '\n';  // TSan capture; no required ordinary output
```

▸ TSan: data race

What goes wrong: The two unsynchronized increments conflict on the same non-atomic memory location without a happens-before relation: a C++17 data race. Undefined behavior does not guarantee lost updates, torn values or a particular count. The fixed unsigned-long atomic count uses defined RMW operations and representable loop/count values.

✓ FIX

```cpp
std::atomic<unsigned long> ticks{0};
std::thread t([&] { for (unsigned long i = 0; i < 100000UL; ++i) ++ticks; });
for (unsigned long i = 0; i < 100000UL; ++i) ++ticks;  // atomic RMW
t.join();
std::cout << ticks << '\n';  // 200000 after both loops complete and join returns
```

Avoid it: Author guidance: use a suitable atomic or a shared locking protocol for these accesses; atomicity of a counter is not a complete publication/lifetime protocol for other data. After both100000-increment loops complete and join returns, the fixed count is200000. Lock freedom is not promised. TSan checks executed instrumented paths and has tool/platform limits.

Sources: C++17 draft 4.7 [intro.races]; 6.9.1 [basic.fundamental]; 32.6 [atomics.types.generic]; 33.3.2.5 [thread.thread.member]; Clang ThreadSanitizer docs Usage; Limitations

CPP-TRAP-09 Virtual call from a constructor

SR-404 · Brakes ECU init

✕ BUGGY

```cpp
struct Ecu {
    Ecu() { init(); }  // runs Ecu::init, not the override
    virtual ~Ecu() = default;
    virtual void init() { std::puts("generic init"); }
};
struct Brakes : Ecu {
    void init() override { std::puts("ABS init"); }
};
```

▸ prints: generic init

What goes wrong: For the call on the object currently in Ecu's constructor, virtual dispatch selects the final overrider in that constructor's class, giving generic init. Construction/destruction has specific dispatch rules; calls on other complete objects differ. A virtual call to a pure virtual for that construction/destruction case is undefined, not a required abort.

✓ FIX

```cpp
struct Ecu {
    virtual ~Ecu() = default;
    virtual void init() { std::puts("generic init"); }
};
struct Brakes : Ecu {
    void init() override { std::puts("ABS init"); }
};
template <class T> std::unique_ptr<T> make_ecu() {
    auto e = std::make_unique<T>();
    e->init();  // fully constructed: dispatches
    return e;
}
```

Sources: C++17 draft 15.7 [class.cdtor]p4; 13.4 [class.abstract]p6; 23.11.1.2.2 [unique.ptr.single.dtor]

Avoid it: Author guidance: choose an initialization API that avoids depending on a not-yet-constructed derived part. This factory calls init only after T is constructed and retains ownership if init throws, but does not roll back arbitrary external side effects. Virtual calls in constructors are not all forbidden by the language.

CPP-TRAP-10 map::operator[] inserts on lookup

SR-404 · Speed-limit table

✕ BUGGY

```cpp
std::map<std::string, int> limits{{"rpm", 6500}};
if (limits["speed"] > 80)           // inserts {"speed", 0}!
    std::puts("too fast");
std::cout << limits.size() << '\n'; // 2
```

▸ prints: 2

What goes wrong: For the absent speed key in this N4659 map, operator[] inserts an entry whose int mapped value is initialized to zero. An existing key does not cause a second insertion. These associative containers have no const operator[] overload in this version; the supplied GCC negative case checks that expression.

✓ FIX

```cpp
std::map<std::string, int> limits{{"rpm", 6500}};
auto it = limits.find("speed");
if (it != limits.end() && it->second > 80)
    std::puts("too fast");
std::cout << limits.size() << '\n'; // 1
```

Avoid it: Author guidance: choose find for an explicit missing-key branch or at for checked access that throws out_of_range on absence. C++20 contains is a separate facility. Protect shared mutations and follow container data-race contracts; insertion-capable lookup is not a general unsynchronized-read API. The fixed find path leaves this map unchanged.

Sources: C++17 draft 26.4.4.3 [map.access]; 26.2.2 [container.requirements.dataraces]; 26.2.6 [associative.reqmts]; C++20 draft N4861 [associative.reqmts]; contains

CPP-TRAP-11 std::move on a const object copies

SR-404 · Trip computer name

× BUGGY

```
const std::string name = "SR-404 TRIP COMPUTER";
std::string dst = std::move(name);   // const&& → copy constructor
std::cout << name.size() << '\n';  // copy selected by these string overloads
```

▸ prints: 20

What goes wrong: Here moving the const string yields const string&&, which cannot bind the ordinary string&& move constructor, so the string copy overload is selected. Other user types can provide different overloads. Equal printed lengths do not by themselves prove copying or buffer transfer. An eligible named local returned as std::move(local) loses NRVO eligibility.

> FIX

```
std::string name = "SR-404 TRIP COMPUTER";  // non-const: movable
std::string dst = std::move(name);  // move constructor selected
std::cout << dst.size() << '\n';
```

Avoid it: Author guidance: use const/value/move according to the intended API rather than applying std::move mechanically. Return an otherwise eligible local by name when NRVO/implicit move is intended. GCC14 documents selected pessimizing/redundant-move warnings. The nonconst string fix selects its move constructor, without promising a particular storage transfer or moved-from value.

Sources: C++17 draft 23.2.5 [forward]; 24.3.2.2 [string.cons]; 15.8.3 [class.copy.elision]; GCC14 C++ options -Wpessimizing-move; -Wredundant-move

Bug examples were compiled; runnable cases were executed on 2026-10-27T00:32:15+0000 with gcc (GCC) 16.2.1 20260810 (C++17). Full sources, including the hidden test drivers: `cheatsheets/src/snippets/<ID>/`. Mistake IDs match `docs/mistakes_catalog.json`, so in-game bugs can link here. Rendering does not repeat compilation. Public drafts are identified in the references; passing checks do not prove all-input correctness. Author correction draft: independent content review remains pending.